GCP Data Engineering Induction Program (40 Days)

Topics	Duration
GCP Cloud Basics	5
Python Basics	3
SQL and BigQuery Analytics	3
Data Warehousing Concepts	2
Spark & PySpark (Dataproc)	6
Apache Airflow (Composer)	3
Pub/Sub and Streaming	2
Cloud Functions & Serverless	2
Data Orchestration (Dataflow)	2
Apache Flink	2
Data Visualization	2
Data Visualization using Tableau	2
Monitoring and Logging	1
GCP SQL Services (Cloud SQL, Spanner, Firestore)	2
Additional Data Engineering Tools	2
Spark Performance Tuning	1
Capstone Project	5
Total	**45**

GCP Cloud Basics (Days 1–5)

Day 1: Introduction to Cloud Computing and Data Engineering

- **Cloud concepts & GCP overview:** Define cloud models (IaaS/PaaS/SaaS) and Google Cloud's global infrastructure
- **Google Cloud console & CLI:** Tour the GCP console and Cloud Shell/SDK. Set up a free-tier project, billing alerts, and budgets. Cover core services (Compute, Storage, Networking, Databases).
- **Introduction to HA & replication:** Explain high availability: multi-zone and multi-region setups.
- **Hands-on:** Create a GCP account, navigate the GCP Console, enable Cloud Shell. Use `gcloud` to list compute zones/regions, and explore IAM roles.

Day 2: Google Compute Services

- **Compute Engine (VMs):** Launch and configure Linux VMs on Compute Engine. Cover instance types, images, startup scripts, and SSH/Cloud Shell access. Discuss use cases for VMs vs containers.
- **Containers & Kubernetes:** Introduce Google Kubernetes Engine (GKE) and Container Registry. Deploy a simple containerized app on GKE. How container orchestration differs from VMs.
- **App Engine:** Overview of App Engine (fully managed serverless).
- **Cloud Functions (intro):** Briefly mention event-driven serverless (Cloud Functions) and Cloud Run for containerized functions
- **Hands-on:** Create a Compute Engine VM, install software. Deploy a container to GKE and expose a service. Optionally deploy a "Hello World" on App Engine.

Day 3: Google Cloud Storage and Databases

- **Cloud Storage:** Dive into Cloud Storage classes (Standard, Nearline, Coldline) and bucket design. Practice uploading and downloading data via the console and `gsutil`.
- **Persistent Disks & File Storage:** Use Persistent Disk with VMs, explore Cloud Filestore for NFS.
- **Cloud SQL (relational DB):** Introduce Cloud SQL (managed MySQL/Postgres). Spin up a Cloud SQL instance; connect via `mysql`/`psql`. Discuss automated backups and read replicas.
- **Cloud Spanner (NewSQL):** Describe Cloud Spanner's globally-distributed SQL. Highlight multi-zone replication and strong consistency
- **NoSQL overview:** Touch on Cloud Bigtable (wide-column store) and Firestore (document DB).
- **Hands-on:** Create a Cloud Storage bucket, upload datasets (e.g. CSV/JSON). Set up a Cloud SQL instance and load sample data. (Demonstrate connecting from Compute VM.)

Day 4: Networking, IAM, and Security

- **Networking fundamentals:** Build a Virtual Private Cloud (VPC) with subnets. Configure firewall rules, routes, and a Cloud VPN. Discuss global vs regional resources.
- **Load balancing & CDN:** Set up a simple HTTP load balancer with a backend service. Explore Cloud CDN for static content caching.
- **Identity and Access Management (IAM):** Explain IAM roles and service accounts. Show how to create custom roles. Discuss principle of least privilege for data engineers.
- **Security & encryption:** Cover default encryption of data at rest. Briefly introduce Cloud KMS (key management) and Shielded VMs for trust.
- **High availability concepts:** Explain multi-zone and multi-region for VPC (e.g. cross-regional load balancing).
- **Hands-on:** Create a VPC network with two subnets and firewall rules. Deploy two VMs behind an HTTP(S) Load Balancer. Experiment with IAM by granting a limited custom role to a new user/service account.

Day 5: GCP Logging, Monitoring and Governance

- **Cloud Monitoring & Logging:** Introduce Cloud Logging (formerly Stackdriver). Create log-based metrics and alerts. Build a Cloud Monitoring dashboard (metrics from GCE, GKE, etc.). Explain how all GCP resources emit logs.
- **Policy and Governance:** Cover organization policies, folders, and projects hierarchy. Demonstrate setting a folder-level policy (e.g. disabling public IPs on VMs).
- **Data Catalog & Data Governance (intro):** Introduce Cloud Data Catalog for metadata management. Show tagging of datasets and IAM on datasets. (Full Data Governance lab on Day 34.)
- **Service Accounts and Access:** Show how to manage service accounts and IAM for automation. Emphasize using service accounts for code access to GCP.
- **Hands-on:** Configure a Monitoring alert. Use Cloud Logging to create a filter and metric. Tag a Cloud Storage bucket or BigQuery dataset and enforce an IAM policy via the Data Catalog/Resource Manager.

Python Programming (Days 6–8)

Day 6: Python Fundamentals for Data Engineering

- **Language Basics:** Cover Python syntax, variables, data types (str, int, float, bool), and arithmetic. Explain control flow: if/else, for/while loops.
- **Data Structures:** Explore lists, tuples, dictionaries, sets. Show common operations (append, slicing, dict lookup).
- **Functions and Modules:** Write Python functions, understand scope, default arguments, and return values. Import standard libraries (os, sys, math). Discuss virtual environments.
- **Hands-on:** Write a script to parse a CSV file and summarize data (e.g. count records). Create and use a custom Python module.

Day 7: Advanced Python and Libraries

- **File I/O and APIs:** Read/write files (CSV, JSON). Access web APIs using `requests`. Introduce Google Cloud client libraries (e.g. `google-cloud-storage`).
- **Error Handling and Logging:** Use `try/except` to handle errors. Add logging via `logging` module. Discuss best practices for debugging and code comments.
- **Object-Oriented Basics:** Introduce simple classes, objects, and methods. Use OOP to encapsulate data processing logic (optional).
- **Hands-on:** Use Python to call a Google API (e.g. list buckets via `google-cloud-storage`). Implement error handling for missing files.

Day 8: Scripting and Automation

- **Scripting for Automation:** Write a Python script to automate a data task (e.g. batch renaming files in Cloud Storage). Use command-line arguments (`argparse`).

- **Working with Data Libraries:** Briefly introduce Pandas for CSV/JSON manipulation (for reference). Use it for quick ETL on local CSV.
- **Version Control and Sharing:** Demonstrate pushing code to GitHub. (While not GCP, this is crucial for code management.)
- **Hands-on:** Create a Python CLI tool that downloads a file from Cloud Storage and processes it. Upload the script to a shared Git repo (or Cloud Source Repositories).

SQL Based Analytics & BigQuery (Days 9–11)

Day 9: Core SQL and BigQuery Basics

- **SQL Review:** Cover SQL DDL/DML: `CREATE TABLE`, `SELECT`, `INSERT`, `UPDATE`, `DELETE`. Use `WHERE`, `GROUP BY`, `HAVING`, and `ORDER BY`. Explain JOINs and set operations.
- **BigQuery Introduction:** Explain BigQuery as a serverless, petabyte-scale data warehouse. Show differences from traditional RDBMS (no index maintenance, separate compute/storage). Emphasize that "BigQuery decouples storage and compute," enabling on-demand scaling
- **Nested and Repeated:** Demonstrate BigQuery's support for nested fields and arrays. Load a JSON dataset with repeated fields and query it using UNNEST.
- **Hands-on:** Create a BigQuery dataset and table. Load sample CSV/JSON data from Cloud Storage. Write SQL queries to join tables and aggregate.

Day 10: Advanced BigQuery Features and Performance

- **Partitioning & Clustering:** Teach date/time partitioned tables and integer range partitions. Use clustering to optimize query performance on large datasets. Demonstrate cost savings from partitions.
- **BigQuery ML & Analytics:** Introduce BigQuery BI Engine and BigQuery ML (machine learning with SQL).
- **Hands-on:** Create partitioned and clustered tables. Run a slow query and then optimize it (e.g. add a filter on partition or cluster column). Use BigQuery ML to run a simple linear regression.

Day 11: SQL Analytics

- SQL Basics: SELECT, WHERE, GROUP BY, ORDER BY
- Data modeling and schema design
- Hands-on: Querying data using BigQuery Web UI

- Joins, Subqueries, CTEs, Window Functions
- Aggregates and analytical patterns
- Hands-on: Data exploration and report generation in BigQuery

Data Warehousing Concepts - 3 Days

Day 12: Data Warehousing Basics

- Introduction to Data Warehousing concepts
- OLTP vs OLAP, ETL vs ELT
- Dimensional modeling basics: facts and dimensions
- Hands-on: Identify facts and dimensions from sample application

- Star schema design principles
- Snowflake schema design and differences
- Hands-on: Design star and snowflake schema from normalized model

Day 13: Application Data to Data warehouse data movement

- Denormalization strategies for reporting
- Surrogate keys, slowly changing dimensions (SCD)
- Perform ETL to extract data in Normalized application schema, convert to Star schema and load the data to a Data warehouse in Start schema in denormalized form
- Hands-on: Convert normalized schema to denormalized schema

Spark & PySpark on Dataproc (Days 14–19)

Day 14: Apache Spark and RDD Fundamentals

- **Spark Overview:** Explain Spark's role in big data (in-memory, distributed). Compare RDDs (Resilient Distributed Datasets) vs. DataFrames.
- **RDD Basics:** Create RDDs from files. Perform basic RDD operations: `map()`, `filter()`, `reduceByKey()`. Demonstrate transformations vs actions.
- **Hands-on:** Use a small text file to perform a WordCount with RDDs. Show caching an RDD and how it affects performance.

Day 15: PySpark DataFrames and Spark SQL

- **DataFrame APIs:** Read data into DataFrames from CSV/Parquet. Use DataFrame transformations (`select`, `withColumn`, `filter`) and built-in functions.
- **Spark SQL:** Register DataFrames as temp tables and run SQL queries via `spark.sql()`. Demonstrate joining DataFrames.
- **Hands-on:** Load a public dataset (e.g. NYC taxi data) into a DataFrame. Write Spark SQL queries for analytics (e.g. top routes, peak hours).

Day 16: Spark SQL Schema and Formats

- **Data Formats:** Read/write data in Parquet, Avro, and ORC. Explain compression and schema evolution.

- **Schema Design:** Discuss best practices for Spark schemas (avoiding `null`, using proper types). Show use of `StructType` to define schema explicitly.
- **Hands-on:** Convert a CSV DataFrame to Parquet on Cloud Storage. Read it back to Spark and verify schema.

Day 17: Spark Data Engineering Patterns

- **Joins & Shuffles:** Demonstrate wide vs narrow dependencies. Use broadcast join for small tables. Explain shuffle and its cost.
- **Partitioning:** Repartition and coalesce for parallelism. Show how to tune number of partitions for performance.
- **Hands-on:** Run a large join and use the Spark UI to observe shuffle. Optimize by broadcasting the small table.

Day 18: Google Cloud Dataproc (Managed Spark)

- **Dataproc Intro:** Show how Dataproc automates cluster creation and management. Explain pricing (1¢/vCPU/minute) and use of preemptible workers for cost saving
- **Cluster Operations:** Create a Dataproc cluster with custom settings (image version, region, zone). Demonstrate setting autoscaling, component gateways (e.g. Jupyter interface).
- **Integration:** Emphasize GCP integrations: Dataproc can easily read/write to BigQuery, Cloud Storage, and Cloud Bigtable.
- **Hands-on:** Use `gcloud dataproc clusters create` to spin up a cluster. SSH into the master node. Submit a sample Spark job (`spark-submit`) to process data in Cloud Storage and save results back to BigQuery or GCS.

Day 19: Spark Job Management and Monitoring

- **Job Submission:** Run PySpark jobs via `gcloud dataproc jobs submit pyspark`. Cover job properties (e.g. setting executor memory, number of executors).
- **Monitoring:** Use the Dataproc jobs API and Web UI to track jobs. Introduce Stackdriver (Cloud Logging/Monitoring) for Spark metrics (executors, tasks).
- **Hands-on:** Submit an ETL Spark job through Dataproc. Introduce a bottleneck (e.g. insufficient executors) and observe cluster behavior. Adjust cluster or job parameters and retry.

Apache Airflow on Cloud Composer (Days 20–22)

Day 20: Airflow Concepts and DAG Design

- **Airflow Introduction:** Cover Directed Acyclic Graphs (DAGs), operators, tasks, and scheduling. Explain Airflow's execution flow: parsing DAGs, queuing tasks, and running tasks via workers.

- **DAG Examples:** Build sample DAGs using Python. Use BashOperator and PythonOperator for simple jobs (e.g. run a Python ETL script).
- **Context and Variables:** Use Airflow Variables and Connections to manage credentials (e.g. a BigQuery connection).
- **Hands-on:** Develop a DAG that orchestrates a multi-step workflow (e.g. Cloud Storage → Dataproc Spark job → BigQuery load).

Day 21: Cloud Composer Environment and Architecture

- **Cloud Composer Overview:** Introduce Cloud Composer as Google's managed Airflow service. Explain environment components: Airflow webserver, schedulers, workers, and metadata DB (Cloud SQL). DAG files live in a Cloud Storage bucket in your project.
- **Composer 3 Architecture:** Use the diagram below to illustrate composer's split between customer and Google-managed projects (tenant project for Airflow services). Show how DAG uploads sync via the environment's bucket.
- **Scaling and Maintenance:** Explain Composer environment versions, minor/major upgrades, and how Composer handles worker scaling.
- **Hands-on:** Create a Composer environment in GCP. Upload a DAG to the Composer bucket and trigger it. View the Airflow UI in the Cloud Console and examine logs.

Day 22: Advanced Airflow and Scheduling

- **Operators & Hooks:** Explore Cloud-specific Airflow operators (e.g. `BigQueryOperator`, `DataprocSubmitJobOperator`, `GCSFileTransformOperator`). Explain how hooks provide connections to GCP services.
- **Task Dependencies and Triggering:** Design complex DAGs with branching, retries, and SLAs. Show time-based scheduling (`@daily`, cron) and event-based triggering (e.g. trigger DAG on Cloud Storage file arrival via Pub/Sub).
- **Monitoring DAG Runs:** Use Airflow's UI to view DAG run history, task logs, and catch scheduling issues. Set up email alerts on DAG failures.
- **Hands-on:** Enhance the previous DAG to include Google-specific operators (e.g. BigQuery and Dataproc jobs). Schedule it to run automatically and test failure/retry logic.

Pub/Sub and Streaming Pipelines (Days 23–24)

Day 23: Google Cloud Pub/Sub Fundamentals

- **Pub/Sub Concepts:** Explain the publish/subscribe messaging pattern. Define topics and subscriptions (push vs pull). Describe message retention and acknowledgment.
- **Scalability & Use Cases:** Highlight Pub/Sub's massive scale: Google built it on infrastructure used by Gmail and Ads to handle over 500 million messages/sec. It provides global access: publishers/subscribers connect to any region, and messages are stored with durability.
- **Security & Delivery:** Discuss authentication (IAM for topics). Show how to set up push to a Google Cloud Function or pull to a subscriber app.

- **Hands-on:** Create a Pub/Sub topic and subscription. Write a simple Python publisher that sends messages, and a subscriber that pulls and processes them. Test message ordering and at-least-once delivery.

Day 24: Streaming Dataflow with Pub/Sub

- **Building Streaming Pipelines:** Introduce Cloud Dataflow (Apache Beam) for stream processing. Show how to read from Pub/Sub in a streaming pipeline and write results to BigQuery or Cloud Storage.
- **Backpressure & Scaling:** Explain how Dataflow autoscaling handles variable message rates. Describe windowing and triggers (Fixed vs Sliding windows).
- **Error Handling:** Demonstrate DLQs (dead-letter topics) for unprocessed messages.
- **Hands-on:** Develop an Apache Beam Python pipeline: read real-time data from Pub/Sub (e.g. clickstream or simulated events), perform a transformation (e.g. count per minute), and write to a BigQuery streaming table. Deploy it on Dataflow in streaming mode. Verify near-real-time results by querying the BigQuery table.

Cloud Functions & Serverless (Days 25–26)

Day 25: Google Cloud Functions and Event-Driven Architecture

- **Cloud Functions Basics:** Cover Cloud Functions (Gen 2 on Cloud Run) for event-driven compute. Show triggers: HTTP, Pub/Sub, Cloud Storage, Firestore, etc.
- **Building Functions:** Write a Python (or Node.js) Cloud Function that responds to a Pub/Sub message or HTTP call. Test locally with Functions Framework and then deploy to GCP.
- **Stateless Design:** Emphasize that Cloud Functions should be stateless and short-lived. Discuss cold starts and concurrency settings.
- **Hands-on:** Create a Cloud Function triggered by a new file in a Cloud Storage bucket (e.g. on file upload, read and log file metadata). Monitor logs via Cloud Logging.

Day 26: Containers with Cloud Run and Serverless Best Practices

- **Cloud Run:** Introduce Cloud Run as a fully managed container platform. Show how to containerize a simple web service (e.g. Flask app) and deploy to Cloud Run. Compare with Cloud Functions (Cloud Run supports any language/RAM beyond functions).
- **Serverless Integration:** Explain how Cloud Functions and Cloud Run integrate with other services (e.g. Cloud Scheduler, Pub/Sub).
- **Performance & Security:** Discuss memory/timeout tuning for Functions, and request/CPU settings for Cloud Run. Cover encryption of environment variables and VPC connectors for private networking.
- **Hands-on:** Write a small containerized API and deploy on Cloud Run. Set up a Cloud Scheduler job to ping it daily. Use IAM to restrict who can invoke the service.

Data Orchestration with Dataflow (Apache Beam) (Days 27–28)

Day 27: Apache Beam Programming Model

- **Beam Concepts:** Explain PCollections, PTransforms, and the unified batch/stream model. Show how Beam's model allows the same code to run on Dataflow or other runners (like Flink).
- **Writing a Pipeline:** Write a simple Beam pipeline in Python (e.g. pipeline that reads text from Cloud Storage, transforms it, and writes to BigQuery). Focus on the Beam programming model (ParDo, DoFn).
- **Runners:** Discuss how to specify the Dataflow runner when executing. Explain requirements file, staging bucket, and pipeline options.
- **Hands-on:** Build and run an Apache Beam pipeline on Dataflow: ingest a static CSV from Cloud Storage, filter and aggregate the data, and store results in BigQuery. Use the Dataflow monitoring UI to view the DAG graph and progress of each stage.

Day 28: Advanced Dataflow Features

- **Batch vs Stream Pipelines:** Contrast batch pipelines (finite data) and streaming pipelines (unbounded, e.g. Pub/Sub). Show exactly-once processing mode and how to enable at-least-once if needed.
- **Dataflow SQL and Templates:** Introduce Dataflow SQL for simple transformations and Dataflow templates for recurring jobs. Show how to trigger a pre-built template (e.g. Pub/Sub to GCS) from the UI or API.
- **Monitoring and Debugging:** Use the Dataflow monitoring interface to profile a job (GPU usage, parallelism). Handle common issues like worker VM errors or hot keys.
- **Hands-on:** Create and run a Dataflow Flex Template from a custom Beam pipeline. Set up a streaming pipeline with windowing and triggers, then generate late data to test trigger logic. Observe metrics via Cloud Monitoring (jobs/views).

Apache Flink on GCP (Days 29–30)

Day 29: Apache Flink Fundamentals

- **Flink Overview:** Describe Flink as a framework for stateful stream and batch processing. Compare Flink's event-time processing with Beam. Cover key Flink components: JobManager and TaskManagers, and how Flink achieves exactly-once.
- **State & Time Semantics:** Explain Flink's stateful streaming and windowing (Watermarks, Event Time windows).
- **Hands-on:** Write a simple Flink program in Java or Python (e.g. WordCount from a socket stream). Run it locally via `flink run`. Observe Flink's web UI to see task parallelism and state.

Day 30: Running Flink on Google Cloud

- **Dataproc Flink Component:** Learn that GCP offers Flink via the Dataproc optional components. Create a Dataproc cluster with Flink enabled. This provides a Flink jobmanager and worker setup.
- **Submitting Flink Jobs:** Submit a Flink job using the Dataproc Jobs API or CLI. Optionally run an Apache Beam pipeline with the Flink runner on the same cluster.
- **High Availability:** Note that Dataproc's High-Availability cluster mode (3 masters) does *not* support Flink HA mode. Explain Flink's own failover mechanisms in that context.
- **Hands-on:** On the Flink-enabled Dataproc cluster, run a streaming Flink job (e.g. count occurrences of words from a Pub/Sub connector). Experiment with taking snapshots for fault tolerance.

Data Visualization (Days 31–32)

Day 31: Looker Studio (Data Studio) Basics

- **Looker Studio Introduction:** Introduce Google's BI tool (formerly Data Studio) for building dashboards on top of BigQuery (or other sources).
- **Connecting Data:** Demonstrate connecting a BigQuery dataset to Looker Studio. Explore dimensions and metrics.
- **Report Design:** Teach how to drag charts, tables, and filters into a report. Explain usage of calculated fields and date filters.
- **Hands-on:** Build a sample dashboard: e.g. sales trends by region using a BigQuery sample dataset. Use charts, scorecards, and data filters. Share the report with teammates.

Day 32: Dashboarding and Analytics

- **Looker (Advanced):** Briefly mention Looker (if available) or data visualization best practices (storytelling with data).
- **Data Studio Features:** Use blended data sources or community visualizations. Show how to schedule email delivery of reports.
- **GCP Integration:** Explain how Looker Studio can visualize real-time data from BigQuery Streaming, or refresh data from GCS/Sheets.
- **Hands-on:** Enhance the dashboard with interactive controls. Optionally, connect a Google Sheets or Cloud SQL data source for combined reporting. Validate that updates in BigQuery appear on the dashboard.

Data Visualization (Tableau) - (Day 33-34)

Day 33:

- Tableau basics and interface overview
- Working with csv & excel files, visualization using Tableau
- Connecting to SQL/BigQuery and other data sources

- Hands-on: Build dashboards using Tableau Desktop

Day 34:

- Filters, calculated fields, parameters
- Publishing dashboards and sharing insights
- Exporting Dashboards, automation
- Hands-on: Build KPI dashboards with filters and drill-downs

Monitoring, Logging & Observability (Day 35)

Day 35: Cloud Monitoring & Operations (Stackdriver)

- **Monitoring Dashboards:** Create custom monitoring dashboards (e.g. CPU load, memory, Pub/Sub throughput). Use basic charts and thresholds.
- **Logging and Tracing:** Demonstrate Cloud Logging for viewing logs from GCE, GKE, and Cloud Functions. Show how to create Logs-based Metrics and set alerts (e.g. error rate threshold).
- **Error Reporting & Debugging:** Introduce Error Reporting (for cloud functions/app errors) and Cloud Profiler (for tracing CPU/memory in Java/Python apps).
- **Auditing and Security:** Explain Cloud Audit Logs for compliance (Data Access logs for BigQuery, etc.). Show how to query audit logs.
- **Hands-on:** Enable a log-based alert on an application (e.g. alert on >5 ERROR logs in 5 minutes). Simulate an error in an app and ensure you receive the alert. Create a Monitoring uptime check for an HTTP service.

GCP Database Services – Cloud SQL, Spanner, Firestore (Days 36–37)

Day 36: Cloud SQL and Cloud Spanner Deep Dive

- **Cloud SQL (Managed RDBMS):** Cover architecture of Cloud SQL (as a managed primary-standby pair in different zones for HA). Explain failover (how standby automatically becomes primary). Discuss read replicas and backup strategies.
- **Spanner (Horizontally-Scalable SQL):** Explain that Cloud Spanner partitions data across nodes and keeps multiple replicas (3 or more) per database for consistency. Show how dual/multi-region configurations maintain 99.999% availability by replicating data across regions.
- **High Availability:** Compare SQL vs Spanner HA: Cloud SQL (zonal HA), Spanner (regional/dual-region replication).
- **Hands-on:** Create a Cloud SQL instance in HA mode and test failover (simulate by stopping one zone). Execute a transaction to show automatic recovery. Set up a Cloud Spanner database in a regional config and run a sample distributed transaction.

Day 37: NoSQL Databases – Firestore and Bigtable

- **Cloud Firestore (Document Store):** Introduce Firestore (Serverless NoSQL DB). Explain that Firestore automatically replicates data across multiple zones/regions for 99.999% availability. Cover Firestore modes (Native vs Datastore).
- **Cloud Bigtable (Wide-Column):** Explain Bigtable's design for low-latency access to large tables (best for time-series or key-value). Discuss single-region vs multi-region instances for HA.
- **Choosing Databases:** Compare when to use Firestore vs Bigtable vs Spanner vs Cloud SQL vs BigQuery (use cases, consistency, query patterns).
- **Hands-on:** Create a Firestore database in native mode. Write/read documents via the console and via a small Python snippet. Create a Bigtable instance and insert/read rows using the Python client.

Additional Data Engineering Tools (Days 38–39)

Day 38: Cloud Data Fusion (Visual ETL)

- **Data Fusion Overview:** Introduce Cloud Data Fusion as a managed, GUI-driven ETL tool built on CDAP. Compare with Airflow (visual vs code orchestration).
- **Building Pipelines:** Use the Data Fusion UI to assemble an ETL pipeline: drag sources (e.g. Cloud Storage), transformations (Wrangler/DataPrep plugins), and sinks (BigQuery).
- **Data Fusion Components:** Explain Wrangler (data cleansing), transformations, and pipeline deployment. Discuss how to schedule/publish pipelines.
- **Hands-on:** In a Cloud Data Fusion instance, build and run a pipeline that reads JSON files from GCS, transforms fields (e.g. flatten nested JSON), and loads into a BigQuery table. Debug any schema mismatches using Wrangler.

Day 39: Data Governance, Catalog, and Security

- **Data Catalog:** Deep dive on Cloud Data Catalog for metadata discovery. Create tags (e.g. PII data tag) and apply them to BigQuery tables. Use Search Catalog to find datasets.
- **Data Security:** Cover DLP API for sensitive data scanning. Show column-level security in BigQuery or policy tags for data classification.
- **Audit and Compliance:** Set up an audit log filter to monitor who accessed what data. Demonstrate IAM conditions on BigQuery (e.g. only users with role X can query a dataset).
- **Hands-on:** Tag a BigQuery table column as "confidential" and create an IAM policy tag for it. Scan a Cloud Storage bucket with DLP API for sensitive info. Review the access audit logs for your Dataflow job.

Spark Performance Tuning (Day 40)

Day 40: Advanced Spark and Cluster Tuning

- **Spark Tuning Principles:** Summarize key tuning techniques: proper memory allocation (`spark.executor.memory`, `spark.driver.memory`), number of partitions, caching frequently-used data, and avoiding skew.
- **Performance Practices:** Show use of the Spark UI and logs to identify bottlenecks (stragglers, garbage collection pauses). Demonstrate caching an RDD/DataFrame to speed up repetitive queries.
- **Dataproc Best Practices:** Explain preemptible workers for cost savings, lifecycle management (automatic cluster shutdown), and choosing the right machine types.
- **Hands-on:** Take an existing Spark job on Dataproc and intentionally degrade its performance (e.g. by reducing executors). Then apply optimizations: increase parallelism, cache intermediate results, and rerun to measure speedup. Review metrics to confirm improvement.

Capstone Project (Days 41–45)

Project Overview: Over five days, participants will design and implement an end-to-end data pipeline on GCP, integrating the services learned. The project should involve data ingestion (batch or streaming), processing (Spark/Dataflow), storage (BigQuery or databases), orchestration (Composer), and visualization.

1. **Day 36:** *Planning & Design.* Form teams and select a real-world scenario (e.g. processing IoT sensor data, or analyzing e-commerce logs). Define data sources, choose services (Pub/Sub vs batch load, Spark vs Dataflow, etc.), and design the architecture diagram.
    - *Hands-on:* Draft the pipeline architecture. Set up any required accounts, permissions, and initial infrastructure (buckets, topics, clusters).
2. **Day 37:** *Data Ingestion & Storage.* Implement the ingestion layer. If streaming, publish test data to Pub/Sub. If batch, load sample files to Cloud Storage. Create Dataflow/Pipeline code or Spark job to bring raw data into a staging store (e.g. Cloud Storage, Cloud SQL, or BigQuery).
    - *Hands-on:* Deploy and run the ingestion pipeline. Verify data arrival in storage (check GCS files, Pub/Sub, or initial database tables).
3. **Day 38:** *Data Processing & Analytics.* Develop the transformation stage. For batch pipelines use Spark on Dataproc; for streaming use Dataflow/Beam or Flink. Perform cleanses, aggregations, or ML feature engineering. Load the processed data into final storage (partitioned BigQuery table or a database).
    - *Hands-on:* Execute the processing jobs. Use Cloud Logging/Monitoring to ensure they complete successfully. Run sample queries on the output data to validate transformations.
4. **Day 39:** *Orchestration & Visualization.* Set up a Composer DAG to automate the workflow (ingest → process → load → report). Build final data models or BI datasets. Create dashboards or reports (in Looker Studio) to visualize key metrics from the processed data. Ensure proper error handling and alerts are in place.

    - *Hands-on:* Trigger the full pipeline via Composer or scheduler. Populate a data visualization and prepare a brief presentation of insights (charts, tables).
5. **Day 40:** *Finalization & Presentation.* Perform end-to-end tests of the pipeline, including a failure recovery test (e.g. restart a component). Finalize documentation and prepare a demonstration. Each team presents their architecture, code, and results, highlighting how they applied GCP services and best practices. Feedback is given for improvements.